ECE2132 Data Structures & Algorithms Lab — To-write (record)

For each experiment: Aim · Theory (summary) · Algorithm · Flowchart · Program · Result. Programs are the corrected, compiled and tested versions; outputs are from real runs on the manual's sample inputs. Software used (all experiments): VS Code / Code::Blocks / Dev-C++ with the GCC/G++ compiler, Windows/Linux.

Exp	Title
1	Pointers
2	Pointer arithmetic
3	Insertion in an array (beginning, any position, end)
4	Deletion from an array (beginning, any position, end)
5	Matrix multiplication using modular programming
6	Matrix transpose using modular programming
7	Insertion sort and its performance
8	Quick sort and comparison with other sorting techniques
9	Stack using an array *(probable: no Exp 9 file was uploaded; confirm before writing)*
10	Queue using an array (enqueue, dequeue)
11	Singly linked list (insertion, deletion, traversal)
12	Circular linked list (insertion, deletion, traversal)

Experiment 1 — Pointers

AIM

To write a program in C++ to implement pointers: store the address of a variable in a pointer and access the variable through the pointer.

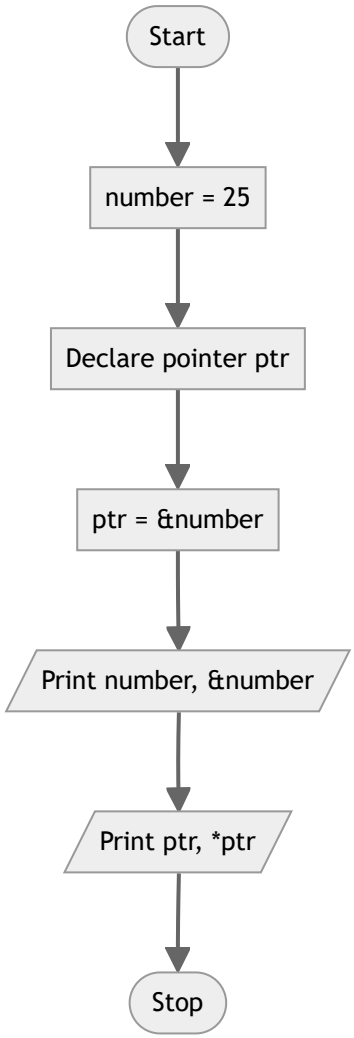
THEORY

A pointer is a variable that stores the address of another variable. `&` gives a variable's address; `*` gives the value at the address a pointer holds. If `ptr = &number`, then `*ptr` equals `number`. Time and space: O(1).

ALGORITHM

1. Start.
2. Declare an integer variable `number = 25`.
3. Declare an integer pointer `ptr`.
4. Store the address of `number` in `ptr` (`ptr = &number`).
5. Display the value of `number` and its address `&number`.
6. Display the address stored in `ptr` and the value `*ptr`.
7. Stop.

FLOWCHART



PROGRAM

E01_POINTERS.CPP

```cpp
#include <iostream>
using namespace std;

int main()
{
    int number = 25;
    int *ptr;              // ptr can hold the address of an int

    ptr = &number;         // & = "address of"

    cout << "Value of number = " << number << endl;
    cout << "Address of number = " << &number << endl;
    cout << "Pointer stores = " << ptr << endl;
    cout << "Value using pointer = " << *ptr << endl;   // * = "value at"

    return 0;
}
```

RESULT

The program was executed successfully. The pointer stored the address of `number` (the same value as `&number`), and dereferencing it (`*ptr`) gave the value 25.

SCREEN (REAL RUN)

```
Value of number = 25
Address of number = 0xbb8cdff8c4
Pointer stores = 0xbb8cdff8c4
Value using pointer = 25
```

Experiment 2 — Pointer arithmetic

AIM

To write a program in C++ to implement pointer arithmetic by performing increment, decrement and addition operations on a pointer.

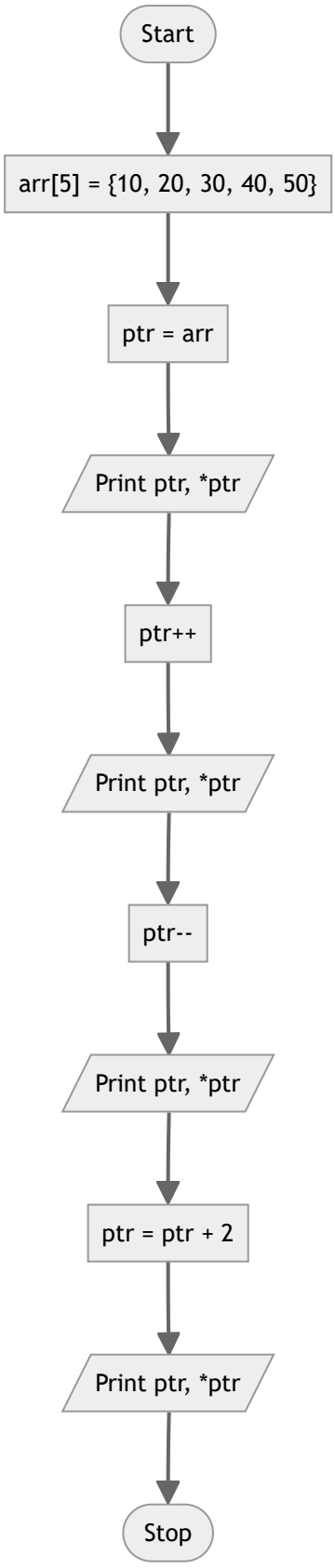
THEORY

Pointer arithmetic moves a pointer by whole elements, not bytes: for an `int` pointer, `ptr++` moves to the next element (+4 bytes), `ptr--` back one, `ptr + 2` forward two. It works because array elements are contiguous, and `arr` is the address of `arr[0]`. Time and space: O(1).

ALGORITHM

1. Start.
2. Declare an integer array `arr[5] = {10, 20, 30, 40, 50}`.
3. Declare a pointer `ptr` and point it to the first element (`ptr = arr`).
4. Display the address in `ptr` and the value `*ptr`.
5. Increment the pointer (`ptr++`); display the address and value.
6. Decrement the pointer (`ptr--`); display the address and value.
7. Add 2 to the pointer (`ptr = ptr + 2`); display the address and value.
8. Stop.

FLOWCHART


Start
arr[5] = {10, 20, 30, 40, 50}
ptr = arr
Print ptr, *ptr
ptr++
Print ptr, *ptr
ptr--
Print ptr, *ptr
ptr = ptr + 2
Print ptr, *ptr
Stop

PROGRAM

E02_POINTER_ARITHMETIC.CPP

```cpp
#include <iostream>
using namespace std;

int main()
{
    int arr[5] = {10, 20, 30, 40, 50};
    int *ptr = arr;                  // array name = address of arr[0]

    cout << "Initial Address : " << ptr << endl;
    cout << "Value : " << *ptr << endl;

    ptr++;                           // moves 1 int forward (4 bytes)
    cout << "\nAfter Increment" << endl;
    cout << "Address : " << ptr << endl;
    cout << "Value : " << *ptr << endl;

    ptr--;                           // moves 1 int back
    cout << "\nAfter Decrement" << endl;
    cout << "Address : " << ptr << endl;
    cout << "Value : " << *ptr << endl;

    ptr = ptr + 2;                   // moves 2 ints forward (8 bytes)
    cout << "\nAfter Adding 2" << endl;
    cout << "Address : " << ptr << endl;
    cout << "Value : " << *ptr << endl;

    return 0;
}
```

RESULT

The program was executed successfully. Each increment moved the pointer by 4 bytes (one int): the values read were 10, 20, 10 and 30 at addresses base, base + 4, base and base + 8.

SCREEN (REAL RUN)

```
Initial Address : 0x45c67ffa90
Value : 10

After Increment
Address : 0x45c67ffa94
Value : 20

After Decrement
Address : 0x45c67ffa90
Value : 10

After Adding 2
Address : 0x45c67ffa98
Value : 30
```

Experiment 3 — Insertion in an array (beginning, any position, end)

AIM

To write a program in C++ to insert an element into an array at the beginning, at any specified position, and at the end.

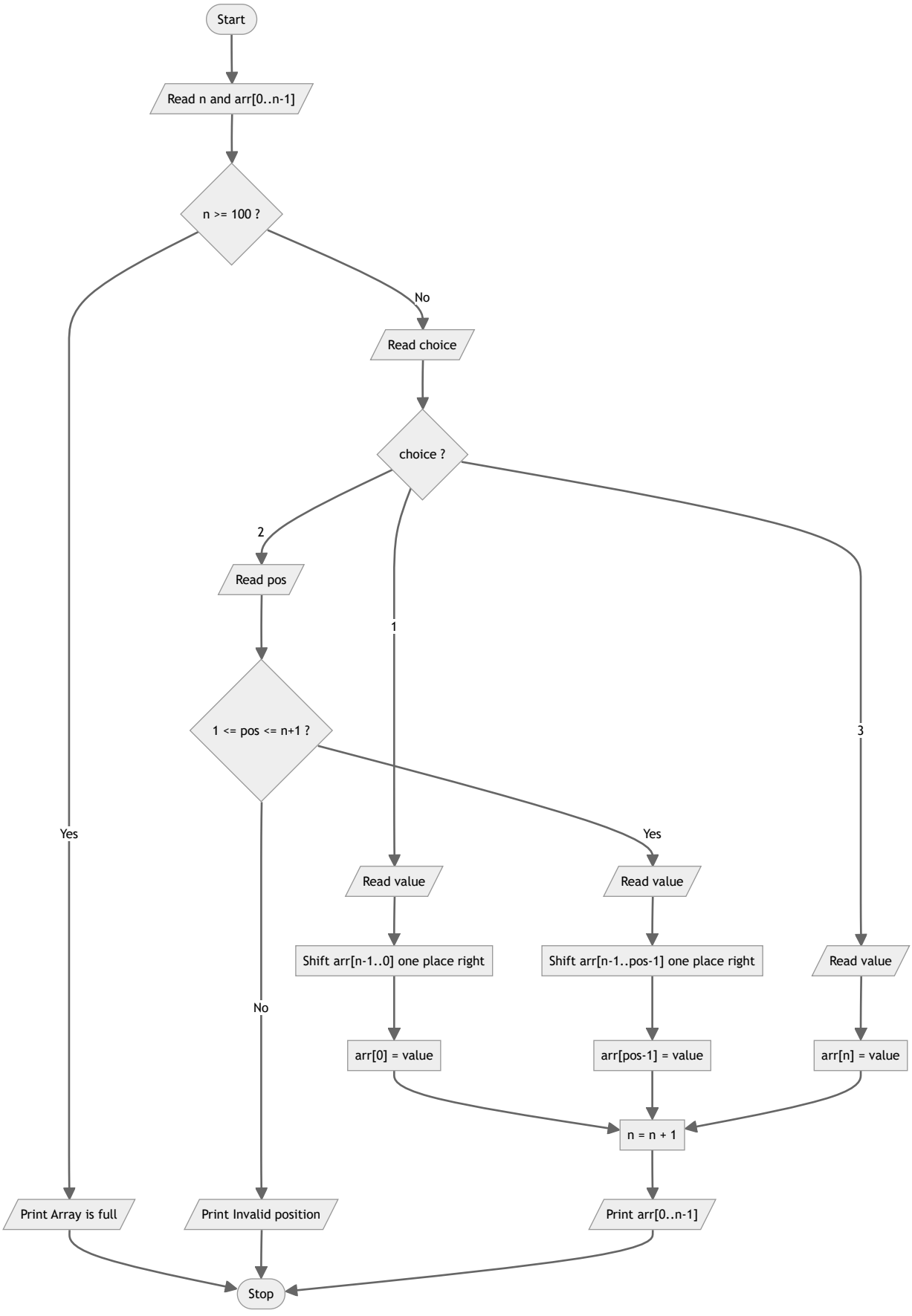
THEORY

Array elements are contiguous, so before inserting, a gap is made by shifting elements one place right, starting from the last. Beginning: shift all, O(n). Any position: shift from index pos−1 onward, O(n). End: no shift, O(1). Then n = n + 1. A full array cannot take more (overflow).

ALGORITHM

1. Start.
2. Declare `arr[100]`; read n and the n elements.
3. If n ≥ 100, display "Array is full" and stop.
4. Display the menu and read the choice.
5. If choice = 1 (beginning): read value; for i = n down to 1, set arr[i] = arr[i−1]; set arr[0] = value.
6. If choice = 2 (position): read pos; if pos < 1 or pos > n+1, display "Invalid position" and stop; read value; for i = n down to pos, set arr[i] = arr[i−1]; set arr[pos−1] = value.
7. If choice = 3 (end): read value; set arr[n] = value.
8. Set n = n + 1.
9. Display the updated array.
10. Stop.

FLOWCHART


Start
Read n and arr[0..n-1]
n >= 100 ?
Yes
No
Read choice
choice ?
1
2
3
Read value
Shift arr[n-1..0] one place right
arr[0] = value
Read pos
1 <= pos <= n+1 ?
Yes
No
Read value
Shift arr[n-1..pos-1] one place right
arr[pos-1] = value
Read value
arr[n] = value
n = n + 1
Print Array is full
Print Invalid position
Print arr[0..n-1]
Stop

PROGRAM

E03_ARRAY_INSERT.CPP

```cpp
#include <iostream>
using namespace std;

int main()
{
    int arr[100], n, value, pos, choice;

    cout << "Enter number of elements: ";
    cin >> n;

    cout << "Enter array elements:\n";
    for (int i = 0; i < n; i++)
        cin >> arr[i];

    if (n >= 100)
    {
        cout << "Array is full. Insertion not possible.";
        return 0;
    }

    cout << "\n1. Insert at Beginning";
    cout << "\n2. Insert at Any Position";
    cout << "\n3. Insert at End";
    cout << "\nEnter your choice: ";
    cin >> choice;

    if (choice == 1)
    {
        cout << "Enter value: ";
        cin >> value;

        for (int i = n; i > 0; i--)          // shift right, last element first
            arr[i] = arr[i - 1];

        arr[0] = value;
        n++;
    }
    else if (choice == 2)
    {
        cout << "Enter Position: ";
        cin >> pos;

        if (pos < 1 || pos > n + 1)
        {
            cout << "Invalid position.";
            return 0;
        }

        cout << "Enter Value: ";
        cin >> value;

        for (int i = n; i >= pos; i--)       // shift right from the end down to pos
            arr[i] = arr[i - 1];

        arr[pos - 1] = value;                // position 3 = index 2
        n++;
    }
    else if (choice == 3)
    {
```

```
        cout << "Enter Value: ";
        cin >> value;

        arr[n] = value;                    // first free slot is index n
        n++;
    }
    else
    {
        cout << "Invalid choice.";
        return 0;
    }

    cout << "\nUpdated Array:\n";
    for (int i = 0; i < n; i++)
        cout << arr[i] << " ";

    return 0;
}
```

RESULT

The program was executed successfully. For 10 20 30 40 50: inserting 5 at the beginning gave 5 10 20 30 40 50; inserting 25 at position 3 gave 10 20 25 30 40 50; inserting 60 at the end gave 10 20 30 40 50 60.

```
SCREEN (REAL RUN)
Enter number of elements: 5
Enter array elements:
10 20 30 40 50

1. Insert at Beginning
2. Insert at Any Position
3. Insert at End
Enter your choice: 2
Enter Position: 3
Enter Value: 25

Updated Array:
10 20 25 30 40 50
```

Experiment 4 — Deletion from an array (beginning, any position, end)

AIM

To write a program in C++ to delete an element from an array at the beginning, at any specified position, and at the end.

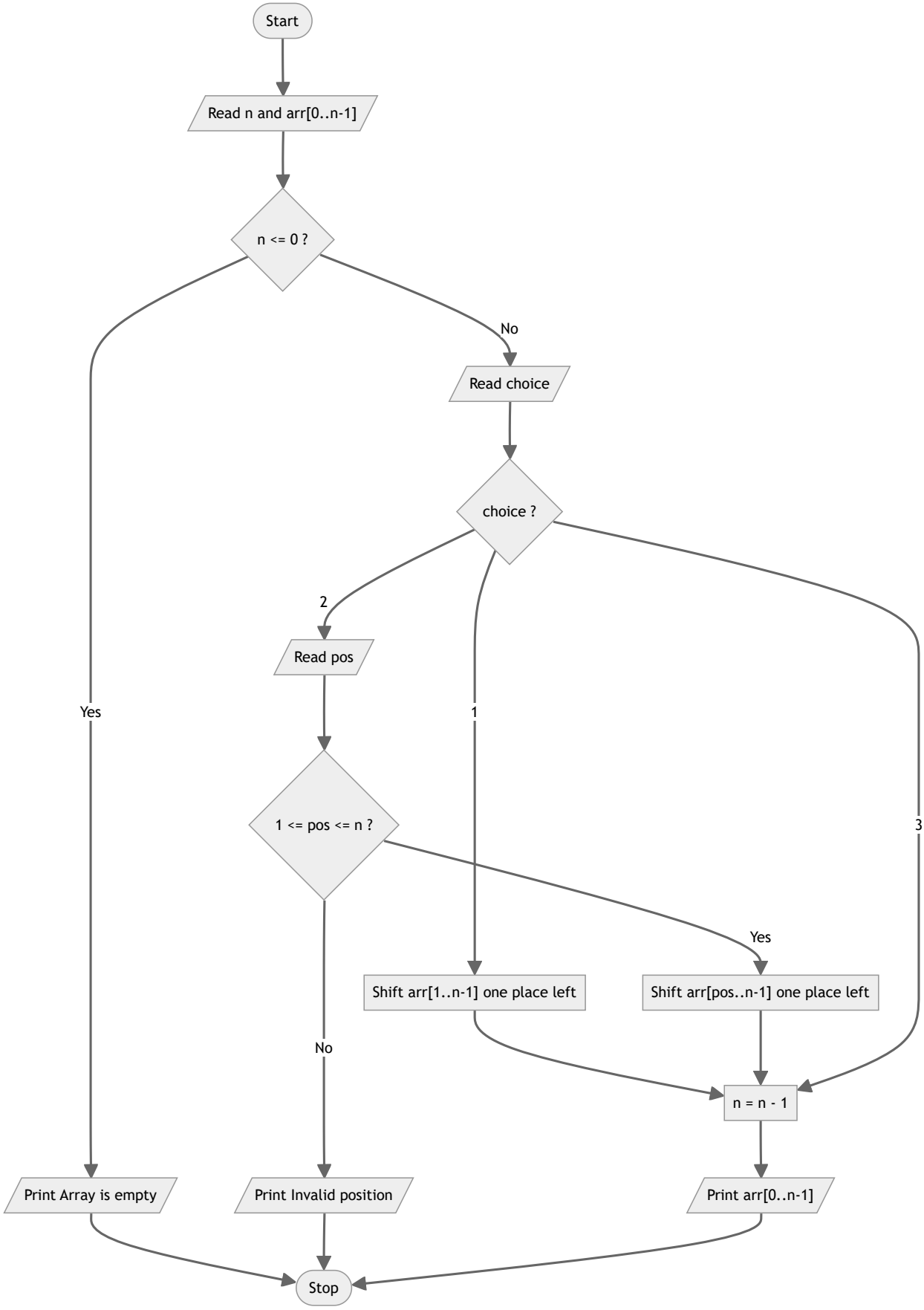
THEORY

Deleting leaves a gap, which is closed by shifting the following elements one place left. Beginning: O(n). Any position: O(n). End: no shift, O(1). Then n = n − 1. Deleting from an empty array is underflow.

ALGORITHM

1. Start.
2. Declare `arr[100]`; read n and the n elements.
3. If n ≤ 0, display "Array is empty" and stop.
4. Display the menu and read the choice.
5. If choice = 1 (beginning): display arr[0]; for i = 0 to n−2, set arr[i] = arr[i+1].
6. If choice = 2 (position): read pos; if pos < 1 or pos > n, display "Invalid position" and stop; display arr[pos−1]; for i = pos−1 to n−2, set arr[i] = arr[i+1].
7. If choice = 3 (end): display arr[n−1].
8. Set n = n − 1.
9. Display the updated array.
10. Stop.

FLOWCHART


Start
Read n and arr[0..n-1]
n <= 0 ?
Yes
No
Read choice
choice ?
1
2
3
Read pos
1 <= pos <= n ?
Yes
No
Shift arr[1..n-1] one place left
Shift arr[pos..n-1] one place left
n = n - 1
Print Array is empty
Print Invalid position
Print arr[0..n-1]
Stop

PROGRAM

E04_ARRAY_DELETE.CPP

```cpp
#include <iostream>
using namespace std;

int main()
{
    int arr[100], n, pos, choice;

    cout << "Enter number of elements: ";
    cin >> n;

    cout << "Enter array elements:\n";
    for (int i = 0; i < n; i++)
        cin >> arr[i];

    if (n <= 0)
    {
        cout << "Array is empty. Deletion not possible.";
        return 0;
    }

    cout << "\n1. Delete at Beginning";
    cout << "\n2. Delete at Any Position";
    cout << "\n3. Delete at End";
    cout << "\nEnter your choice: ";
    cin >> choice;

    if (choice == 1)
    {
        cout << "Deleted element: " << arr[0];
        for (int i = 0; i < n - 1; i++)      // shift left, first element first
            arr[i] = arr[i + 1];
        n--;
    }
    else if (choice == 2)
    {
        cout << "Enter Position: ";
        cin >> pos;

        if (pos < 1 || pos > n)
        {
            cout << "Invalid position.";
            return 0;
        }

        cout << "Deleted element: " << arr[pos - 1];
        for (int i = pos - 1; i < n - 1; i++)
            arr[i] = arr[i + 1];
        n--;
    }
    else if (choice == 3)
    {
        cout << "Deleted element: " << arr[n - 1];
        n--;                                 // just forget the last one
    }
    else
    {
        cout << "Invalid choice.";
        return 0;
    }
```

```
    cout << "\nUpdated Array:\n";
    for (int i = 0; i < n; i++)
        cout << arr[i] << " ";

    return 0;
}
```

RESULT

The program was executed successfully. For 10 20 30 40 50: deleting at the beginning gave 20 30 40 50; deleting position 3 (element 30) gave 10 20 40 50; deleting at the end gave 10 20 30 40.

SCREEN (REAL RUN)

```
Enter number of elements: 5
Enter array elements:
10 20 30 40 50

1. Delete at Beginning
2. Delete at Any Position
3. Delete at End
Enter your choice: 2
Enter Position: 3
Deleted element: 30
Updated Array:
10 20 40 50
```

Experiment 5 — Matrix multiplication using modular programming

AIM

To write a program in C++ to implement matrix multiplication using modular programming, with separate functions for input, multiplication and display.

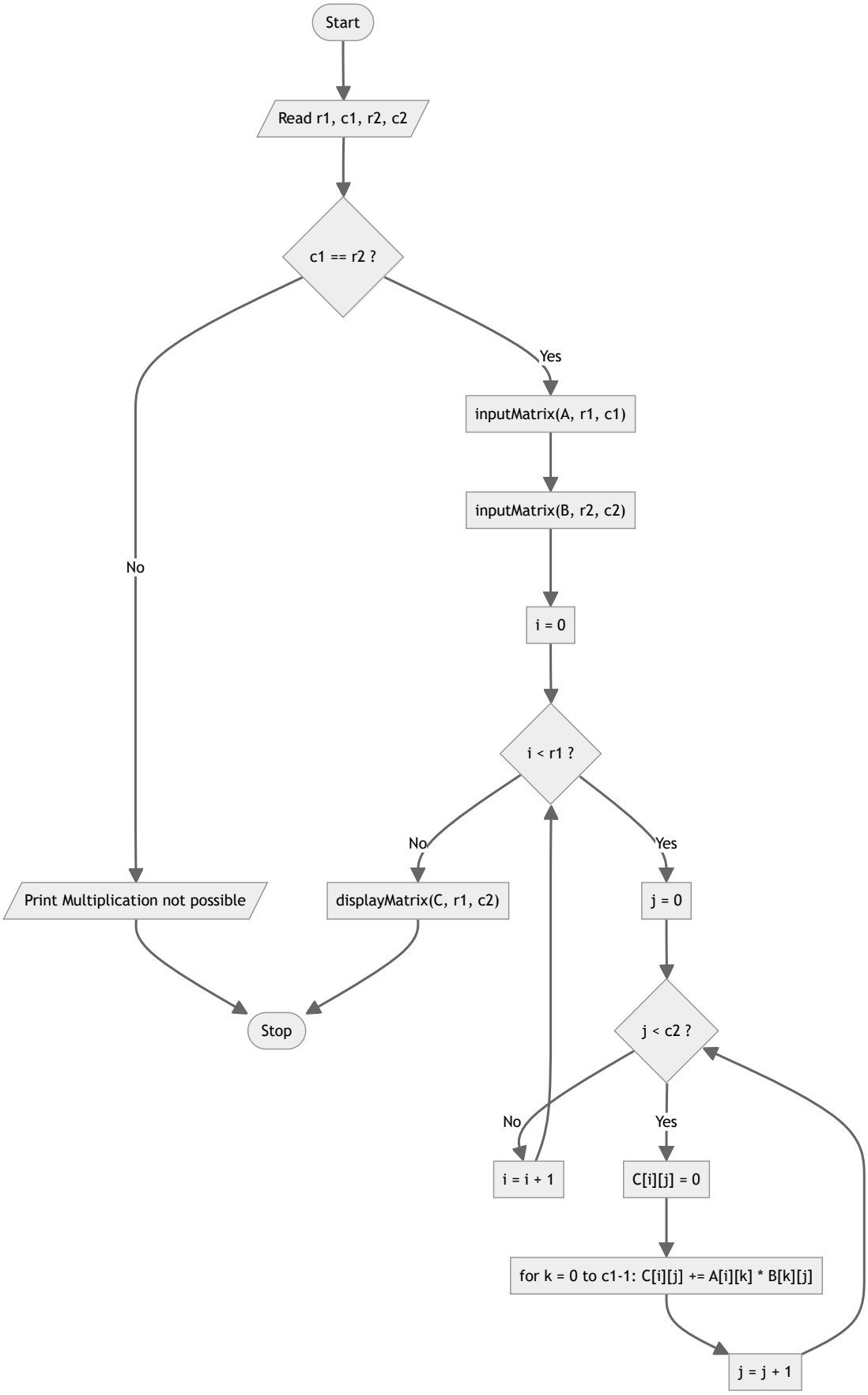
THEORY

A (r1 × c1) and B (r2 × c2) can be multiplied only if c1 = r2. The result is r1 × c2, with C[i][j] = Σ A[i][k] × B[k][j]. Three nested loops give $O(n^3)$ time. Modular programming splits the work into `inputMatrix()`, `multiplyMatrix()` and `displayMatrix()`, called from `main()`.

ALGORITHM

1. Start.
2. Declare matrices A, B, C of size 10 × 10.
3. Read the order r1 × c1 of A and r2 × c2 of B.
4. If c1 ≠ r2, display "Matrix multiplication not possible" and stop.
5. Call `inputMatrix()` to read A and then B.
6. Call `multiplyMatrix()`: for each i (0 to r1−1) and j (0 to c2−1), set C[i][j] = 0, then for k = 0 to c1−1 add A[i][k] × B[k][j] to C[i][j].
7. Call `displayMatrix()` to display C (r1 × c2).
8. Stop.

FLOWCHART


Start
Read r1, c1, r2, c2
c1 == r2 ?
Yes
No
inputMatrix(A, r1, c1)
inputMatrix(B, r2, c2)
i = 0
i < r1 ?
No
Yes
Print Multiplication not possible
displayMatrix(C, r1, c2)
j = 0
Stop
j < c2 ?
No
Yes
i = i + 1
C[i][j] = 0
for k = 0 to c1-1: C[i][j] += A[i][k] * B[k][j]
j = j + 1

PROGRAM

E05_MATRIX_MULTIPLY.CPP

```cpp
#include <iostream>
using namespace std;

// Function to input matrix elements
void inputMatrix(int matrix[][10], int rows, int cols)
{
    for (int i = 0; i < rows; i++)
        for (int j = 0; j < cols; j++)
            cin >> matrix[i][j];
}

// Function to multiply two matrices: C = A x B
void multiplyMatrix(int A[][10], int B[][10], int C[][10], int r1, int c1, int c2)
{
    for (int i = 0; i < r1; i++)              // each row of A
    {
        for (int j = 0; j < c2; j++)          // each column of B
        {
            C[i][j] = 0;
            for (int k = 0; k < c1; k++)      // walk along row i and column j
                C[i][j] = C[i][j] + A[i][k] * B[k][j];
        }
    }
}

// Function to display matrix
void displayMatrix(int matrix[][10], int rows, int cols)
{
    for (int i = 0; i < rows; i++)
    {
        for (int j = 0; j < cols; j++)
            cout << matrix[i][j] << " ";
        cout << endl;
    }
}

int main()
{
    int A[10][10], B[10][10], C[10][10];
    int r1, c1, r2, c2;

    cout << "Enter rows and columns of first matrix: ";
    cin >> r1 >> c1;

    cout << "Enter rows and columns of second matrix: ";
    cin >> r2 >> c2;

    if (r1 < 1 || c1 < 1 || r2 < 1 || c2 < 1 || r1 > 10 || c1 > 10 || r2 > 10 || c2 > 10)
    {
        cout << "Sizes must be between 1 and 10.";
        return 0;
    }

    // Check multiplication condition
    if (c1 != r2)
    {
        cout << "Matrix multiplication not possible.";
        return 0;
    }
```

```
    cout << "Enter elements of first matrix:\n";
    inputMatrix(A, r1, c1);

    cout << "Enter elements of second matrix:\n";
    inputMatrix(B, r2, c2);

    multiplyMatrix(A, B, C, r1, c1, c2);

    cout << "\nResultant Matrix:\n";
    displayMatrix(C, r1, c2);

    return 0;
}
```

RESULT

The program was executed successfully. [[1, 2], [3, 4]] × [[5, 6], [7, 8]] = [[19, 22], [43, 50]], computed with separate functions for input, multiplication and display.

SCREEN (REAL RUN)

```
Enter rows and columns of first matrix: 2 2
Enter rows and columns of second matrix: 2 2
Enter elements of first matrix:
1 2
3 4
Enter elements of second matrix:
5 6
7 8

Resultant Matrix:
19 22
43 50
```

Experiment 6 — Matrix transpose using modular programming

AIM

To write a program in C++ to find the transpose of a matrix using modular programming, with separate functions for input, transpose and display.

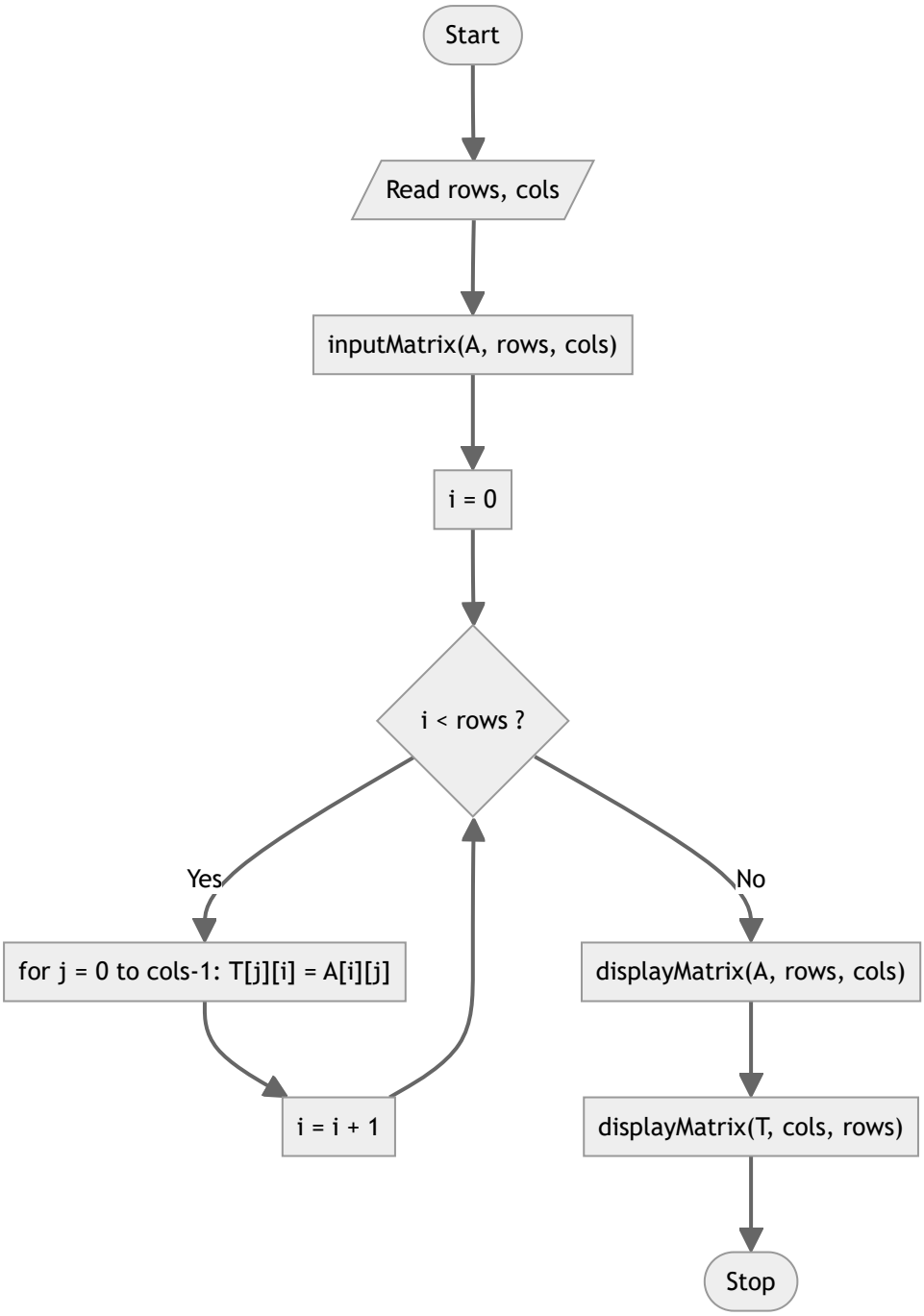
THEORY

The transpose swaps rows and columns: an m × n matrix A gives an n × m matrix T with T[j][i] = A[i][j]. Time and space: O(m × n). Separate functions: `inputMatrix()`, `transposeMatrix()`, `displayMatrix()`.

ALGORITHM

1. Start.
2. Declare matrices A and T of size 10 × 10.
3. Read the number of rows and columns of A.
4. Call `inputMatrix()` to read A.
5. Call `transposeMatrix()`: for each i (0 to rows−1) and j (0 to cols−1), set T[j][i] = A[i][j].
6. Call `displayMatrix(A, rows, cols)` to display the original matrix.
7. Call `displayMatrix(T, cols, rows)` to display the transpose.
8. Stop.

FLOWCHART



PROGRAM

E06_MATRIX_TRANSPOSE.CPP

```cpp
#include <iostream>
using namespace std;

// Function to input matrix elements
void inputMatrix(int matrix[][10], int rows, int cols)
{
    for (int i = 0; i < rows; i++)
        for (int j = 0; j < cols; j++)
            cin >> matrix[i][j];
}
```

```cpp
// Function to find transpose of matrix
void transposeMatrix(int A[][10], int T[][10], int rows, int cols)
{
    for (int i = 0; i < rows; i++)
        for (int j = 0; j < cols; j++)
            T[j][i] = A[i][j];              // row i becomes column i
}

// Function to display matrix
void displayMatrix(int matrix[][10], int rows, int cols)
{
    for (int i = 0; i < rows; i++)
    {
        for (int j = 0; j < cols; j++)
            cout << matrix[i][j] << " ";
        cout << endl;
    }
}

int main()
{
    int A[10][10], T[10][10];
    int rows, cols;

    cout << "Enter number of rows and columns of matrix: ";
    cin >> rows >> cols;

    if (rows < 1 || cols < 1 || rows > 10 || cols > 10)
    {
        cout << "Sizes must be between 1 and 10.";
        return 0;
    }

    cout << "Enter elements of matrix:\n";
    inputMatrix(A, rows, cols);

    transposeMatrix(A, T, rows, cols);

    cout << "\nOriginal Matrix:\n";
    displayMatrix(A, rows, cols);

    cout << "\nTranspose Matrix:\n";
    displayMatrix(T, cols, rows);           // rows and columns swap

    return 0;
}
```

RESULT

The program was executed successfully. The 2 × 3 matrix [[1, 2, 3], [4, 5, 6]] gave the 3 × 2 transpose [[1, 4], [2, 5], [3, 6]].

SCREEN (REAL RUN)

```
Enter number of rows and columns of matrix: 2 3
Enter elements of matrix:
1 2 3
4 5 6

Original Matrix:
1 2 3
4 5 6
```

```
Transpose Matrix:
1 4
2 5
3 6
```

Experiment 7 — Insertion sort and its performance

AIM

To write a program in C++ to sort an array of integers in ascending order using insertion sort and analyse its performance.

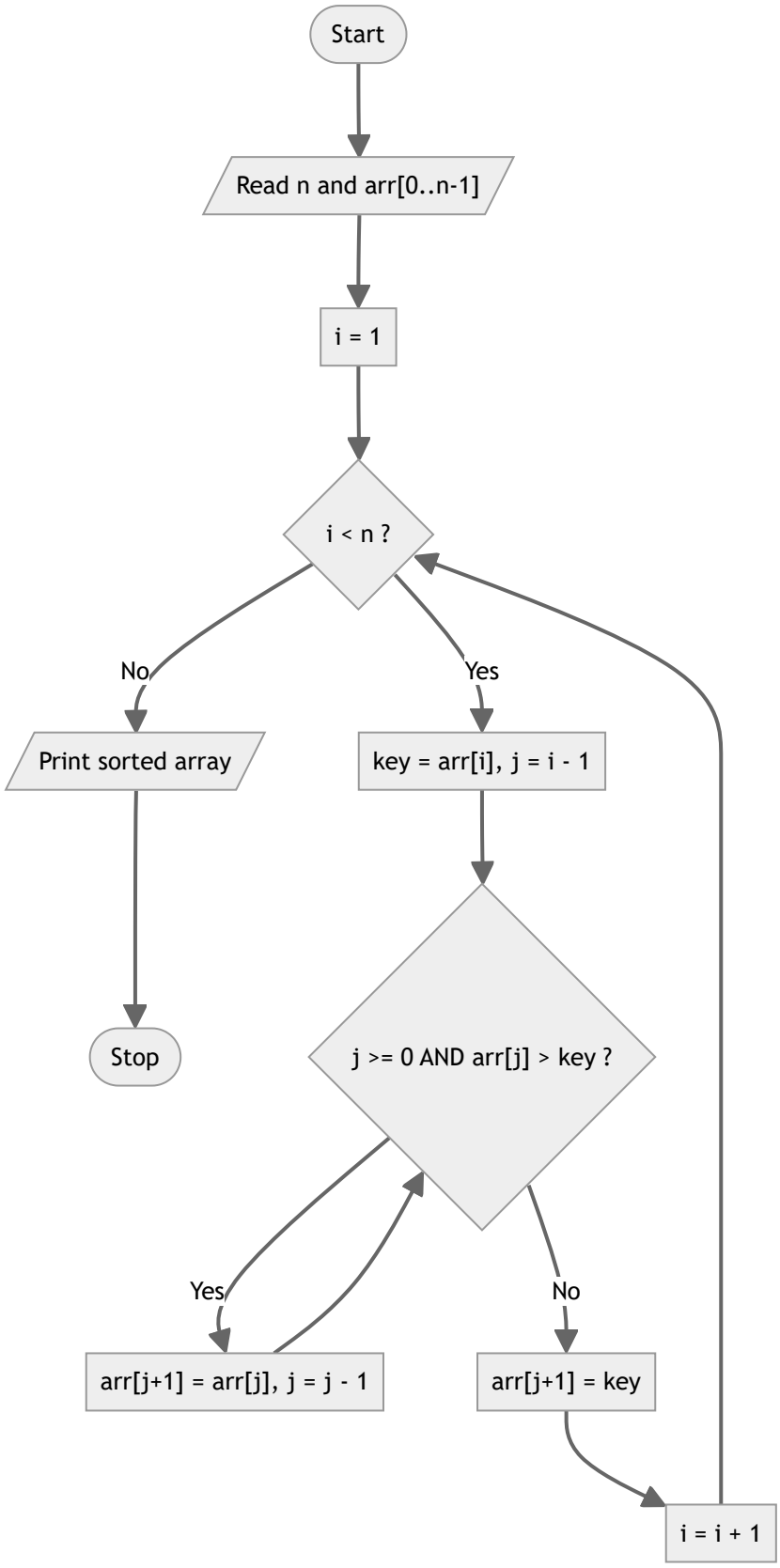
THEORY

Insertion sort keeps a sorted left part. Each pass takes the next element as the key, shifts larger sorted elements one place right, and inserts the key into the gap (n − 1 passes). Best case O(n) (already sorted); average and worst $O(n^2)$ (reverse sorted). Space O(1); in-place and stable.

ALGORITHM

1. Start.
2. Read n and the n elements of the array.
3. For i = 1 to n − 1: a. key = arr[i], j = i − 1. b. While j ≥ 0 and arr[j] > key: arr[j+1] = arr[j], j = j − 1. c. arr[j+1] = key.
4. Display the sorted array.
5. Stop.

FLOWCHART



PROGRAM

E07_INSERTION_SORT.CPP

```cpp
#include <iostream>
using namespace std;
```

```cpp
// Function to perform Insertion Sort
void insertionSort(int arr[], int n)
{
    for (int i = 1; i < n; i++)
    {
        int key = arr[i];                    // card picked up
        int j = i - 1;

        // Shift elements greater than key one position to the right
        while (j >= 0 && arr[j] > key)
        {
            arr[j + 1] = arr[j];
            j--;
        }

        arr[j + 1] = key;                    // drop the card in the gap
    }
}

// Function to display the array
void displayArray(int arr[], int n)
{
    for (int i = 0; i < n; i++)
        cout << arr[i] << " ";
    cout << endl;
}

int main()
{
    int n;
    int arr[100];

    cout << "Enter number of elements: ";
    cin >> n;

    if (n < 1 || n > 100)
    {
        cout << "n must be between 1 and 100.";
        return 0;
    }

    cout << "Enter elements of the array:" << endl;
    for (int i = 0; i < n; i++)
        cin >> arr[i];

    cout << "Original Array: ";
    displayArray(arr, n);

    insertionSort(arr, n);

    cout << "Sorted Array: ";
    displayArray(arr, n);

    return 0;
}
```

RESULT

The program was executed successfully. The array 5 2 4 6 1 3 was sorted to 1 2 3 4 5 6 in 5 passes (9 shifts, 12 comparisons). Performance: best case O(n) (sorted input: 5 comparisons for n = 6), average and worst case $O(n^2)$ (reverse input: 15 comparisons = n(n−1)/2), space O(1).

SCREEN (REAL RUN)

```
Enter number of elements: 6
Enter elements of the array:
5 2 4 6 1 3
Original Array: 5 2 4 6 1 3
Sorted Array: 1 2 3 4 5 6
```

Experiment 8 — Quick sort and comparison with other sorting techniques

AIM

To write a program in C++ to sort an array of integers using quick sort and compare its performance with other sorting techniques (bubble sort, selection sort, insertion sort).

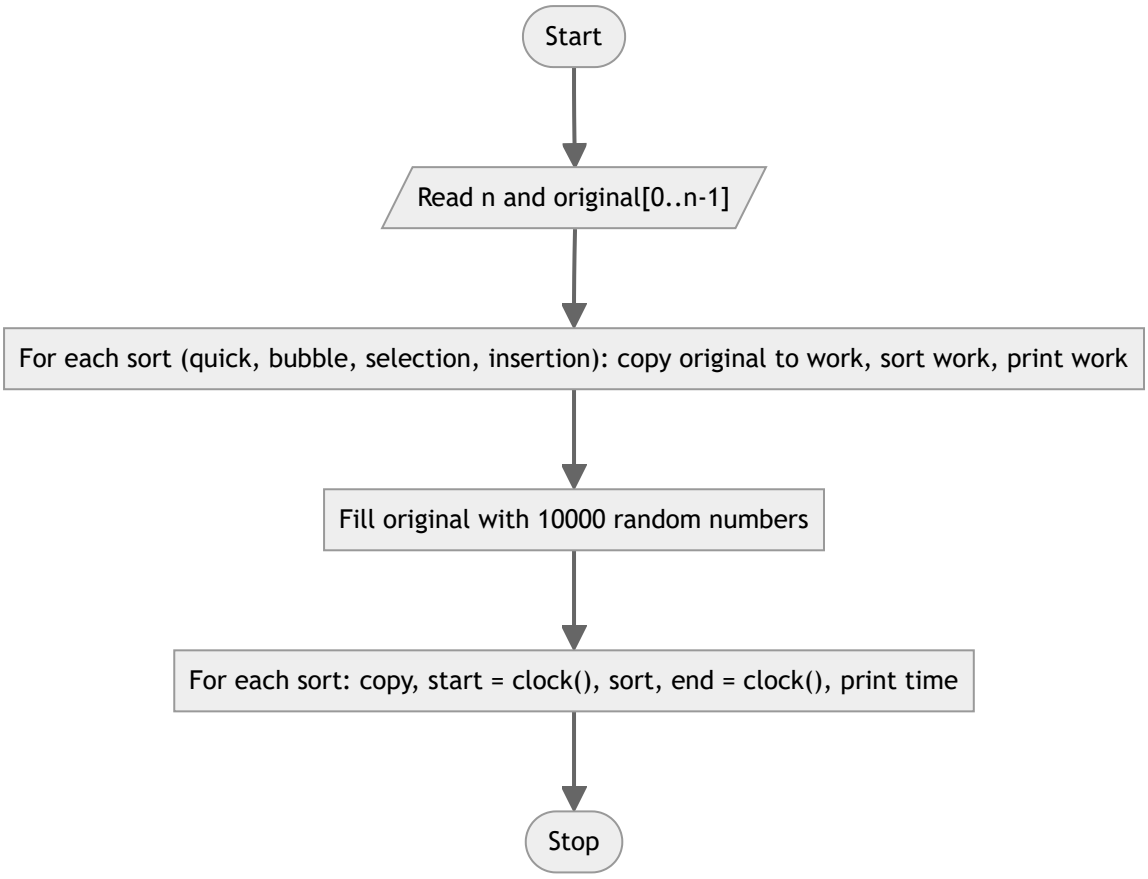
THEORY

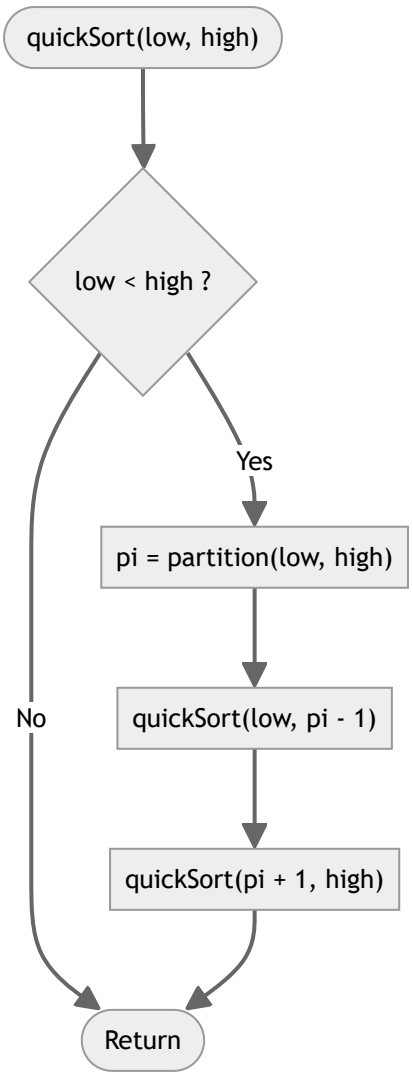
Quick sort (divide and conquer) picks a pivot (the last element) and partitions the array: smaller elements go to the left, larger to the right, and the pivot lands in its final position. Both sides are then sorted recursively. Best/average O(n log n), worst $O(n^2)$ (sorted input), space O(log n). Bubble, selection and insertion sort are $O(n^2)$; execution times are compared using `clock()`.

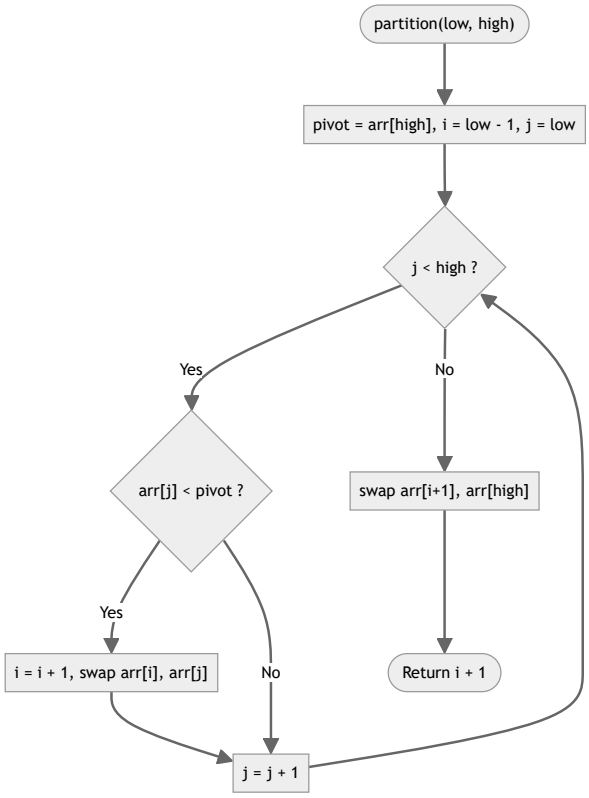
ALGORITHM

1. Start.
2. Read n and the n elements into `original[]`.
3. For each of quick, bubble, selection and insertion sort: copy `original[]` into `work[]`, sort `work[]`, and display it.
4. **quickSort(arr, low, high):** if low < high: pi = partition(arr, low, high); quickSort(arr, low, pi − 1); quickSort(arr, pi + 1, high).
5. **partition(arr, low, high):** pivot = arr[high], i = low − 1; for j = low to high − 1: if arr[j] < pivot then i = i + 1 and swap arr[i], arr[j]. Finally swap arr[i+1], arr[high] and return i + 1.
6. Fill `original[]` with 10 000 random numbers. For each algorithm: copy, record `start = clock()`, sort, record `end = clock()`, display (end − start) in ms.
7. Stop.

FLOWCHART


Start
Read n and original[0..n-1]
For each sort (quick, bubble, selection, insertion): copy original to work, sort work, print work
Fill original with 10000 random numbers
For each sort: copy, start = clock(), sort, end = clock(), print time
Stop





PROGRAM

E08_SORT_COMPARE.CPP

```
#include <iostream>
#include <cstdlib>     // rand, srand
#include <ctime>       // clock, CLOCKS_PER_SEC
using namespace std;

const int BIG = 10000;                    // size used for the timing test

void swapElements(int &a, int &b)
{
    int temp = a;
    a = b;
    b = temp;
}

int partitionArray(int arr[], int low, int high)
{
    int pivot = arr[high];
    int i = low - 1;
    for (int j = low; j < high; j++)
        if (arr[j] < pivot)
        {
            i++;
            swapElements(arr[i], arr[j]);
```

```
        }
    swapElements(arr[i + 1], arr[high]);
    return i + 1;
}

void quickSort(int arr[], int low, int high)
{
    if (low < high)
    {
        int pi = partitionArray(arr, low, high);
        quickSort(arr, low, pi - 1);
        quickSort(arr, pi + 1, high);
    }
}

// Bubble Sort: neighbours swap until the largest "bubbles" to the end
void bubbleSort(int arr[], int n)
{
    for (int i = 0; i < n - 1; i++)
    {
        bool swapped = false;
        for (int j = 0; j < n - 1 - i; j++)
            if (arr[j] > arr[j + 1])
            {
                swapElements(arr[j], arr[j + 1]);
                swapped = true;
            }
        if (!swapped)                       // no swap in a pass = already sorted
            break;
    }
}

// Selection Sort: pick the smallest of the rest, put it at position i
void selectionSort(int arr[], int n)
{
    for (int i = 0; i < n - 1; i++)
    {
        int minIndex = i;
        for (int j = i + 1; j < n; j++)
            if (arr[j] < arr[minIndex])
                minIndex = j;
        swapElements(arr[i], arr[minIndex]);
    }
}

void insertionSort(int arr[], int n)
{
    for (int i = 1; i < n; i++)
    {
        int key = arr[i];
        int j = i - 1;
        while (j >= 0 && arr[j] > key)
        {
            arr[j + 1] = arr[j];
            j--;
        }
        arr[j + 1] = key;
    }
}

void copyArray(int source[], int dest[], int n)
{
    for (int i = 0; i < n; i++)
```

```
        dest[i] = source[i];
}

void displayArray(int arr[], int n)
{
    for (int i = 0; i < n; i++)
        cout << arr[i] << " ";
    cout << endl;
}

int original[BIG], work[BIG];               // global: too big for the stack

int main()
{
    int n;
    cout << "Enter number of elements: ";
    cin >> n;

    if (n < 1 || n > 100)
    {
        cout << "n must be between 1 and 100.";
        return 0;
    }

    cout << "Enter elements of the array:" << endl;
    for (int i = 0; i < n; i++)
        cin >> original[i];

    // Part 1: every algorithm sorts the SAME input (each gets its own copy)
    copyArray(original, work, n);  quickSort(work, 0, n - 1);
    cout << "Quick Sort     : ";  displayArray(work, n);
    copyArray(original, work, n);  bubbleSort(work, n);
    cout << "Bubble Sort    : ";  displayArray(work, n);
    copyArray(original, work, n);  selectionSort(work, n);
    cout << "Selection Sort : ";  displayArray(work, n);
    copyArray(original, work, n);  insertionSort(work, n);
    cout << "Insertion Sort : ";  displayArray(work, n);

    // Part 2: timing needs a big array - 6 numbers sort in ~0 ms
    srand(1);                                // fixed seed: same "random" data every run
    for (int i = 0; i < BIG; i++)
        original[i] = rand() % 100000;

    cout << "\nTime to sort " << BIG << " random numbers:" << endl;
    clock_t start, end;

    copyArray(original, work, BIG);
    start = clock();  quickSort(work, 0, BIG - 1);  end = clock();
    cout << "Quick Sort     : " << 1000.0 * (end - start) / CLOCKS_PER_SEC << " ms" << endl;

    copyArray(original, work, BIG);
    start = clock();  bubbleSort(work, BIG);  end = clock();
    cout << "Bubble Sort    : " << 1000.0 * (end - start) / CLOCKS_PER_SEC << " ms" << endl;

    copyArray(original, work, BIG);
    start = clock();  selectionSort(work, BIG);  end = clock();
    cout << "Selection Sort : " << 1000.0 * (end - start) / CLOCKS_PER_SEC << " ms" << endl;

    copyArray(original, work, BIG);
    start = clock();  insertionSort(work, BIG);  end = clock();
    cout << "Insertion Sort : " << 1000.0 * (end - start) / CLOCKS_PER_SEC << " ms" << endl;
```

```
    return 0;
}
```

RESULT

The program was executed successfully. The array 10 7 8 9 1 5 was sorted to 1 5 7 8 9 10 by all four algorithms. On 10 000 random numbers, quick sort was the fastest by far (about 1 ms), while bubble, selection and insertion sort took tens to over a hundred milliseconds, consistent with O(n log n) vs $O(n^2)$. Times vary from machine to machine.

SCREEN (REAL RUN)

```
Enter number of elements: 6
Enter elements of the array:
10 7 8 9 1 5
Quick Sort     : 1 5 7 8 9 10
Bubble Sort    : 1 5 7 8 9 10
Selection Sort : 1 5 7 8 9 10
Insertion Sort : 1 5 7 8 9 10

Time to sort 10000 random numbers:
Quick Sort     : 1 ms
Bubble Sort    : 114 ms
Selection Sort : 53 ms
Insertion Sort : 37 ms
```

Experiment 9 — Stack using an array *(probable)*

> **CHECK FIRST**
> No Exp 9 file was uploaded. Exp 10's figures say "Operation on Stack", so Exp 9 is most likely Stack. Confirm with a classmate before writing this one.

AIM

To write a program in C++ to implement a stack using an array with push, pop, peek and display operations.

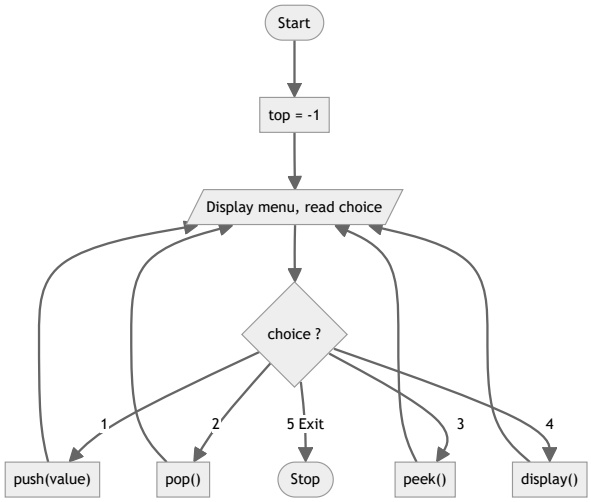
THEORY

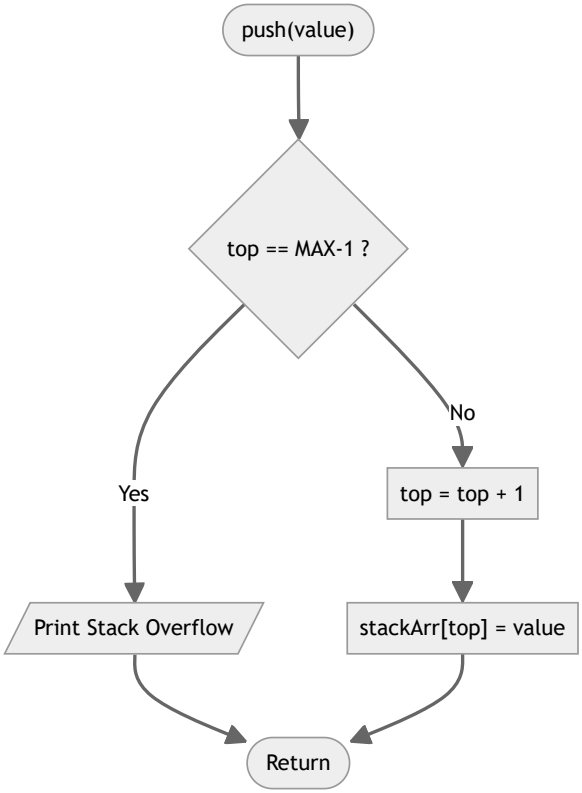
A stack inserts (push) and deletes (pop) at one end, the top: LIFO (Last In, First Out). top = −1 means empty. Overflow: push when top = MAX − 1. Underflow: pop when top = −1. Push, pop and peek are O(1).

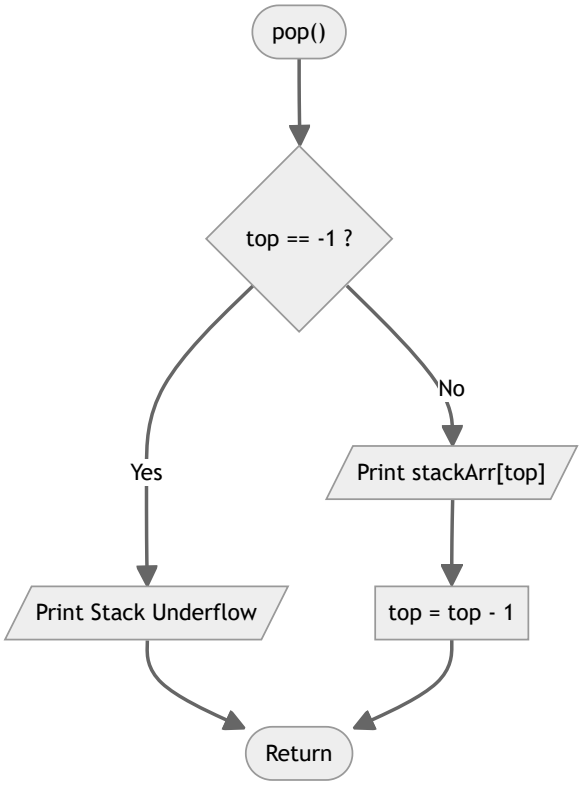
ALGORITHM

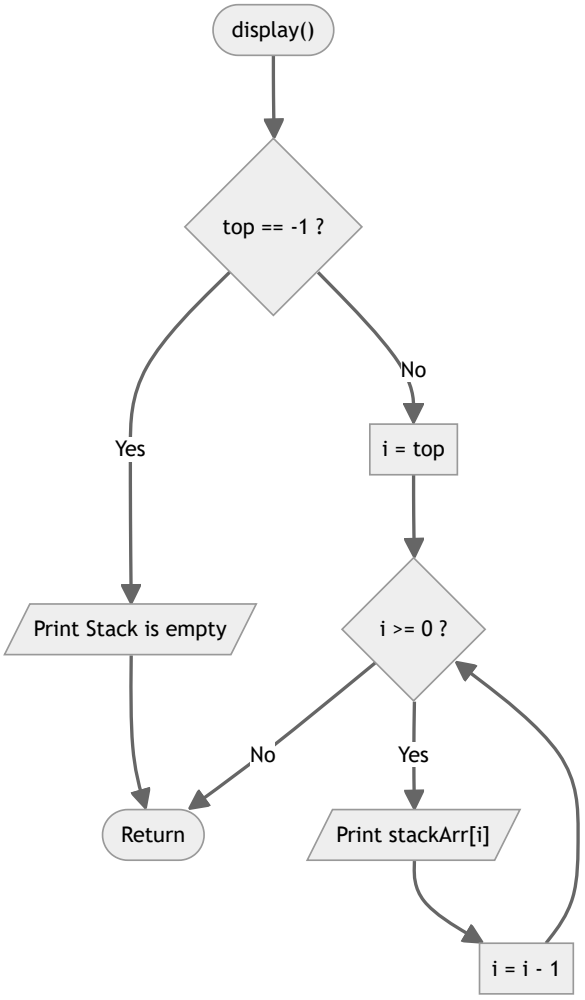
1. Start. Declare `stackArr[MAX]`, top = −1.
2. Display the menu (Push, Pop, Peek, Display, Exit) and read the choice.
3. **Push:** if top = MAX − 1, display "Stack Overflow"; else top = top + 1, stackArr[top] = value.
4. **Pop:** if top = −1, display "Stack Underflow"; else display stackArr[top], top = top − 1.
5. **Peek:** if top = −1, display "Stack is empty"; else display stackArr[top].
6. **Display:** print stackArr[top] down to stackArr[0].
7. Repeat from step 2 until Exit is chosen.
8. Stop.

FLOWCHART


Start
top = -1
Display menu, read choice
choice ?
1
2
5 Exit
3
4
push(value)
pop()
Stop
peek()
display()


push(value)
top == MAX-1 ?
Yes
No
Print Stack Overflow
top = top + 1
stackArr[top] = value
Return


pop()
top == -1 ?
Yes
No
Print Stack Underflow
Print stackArr[top]
top = top - 1
Return


display()
top == -1 ?
Yes
No
Print Stack is empty
i = top
i >= 0 ?
No
Yes
Return
Print stackArr[i]
i = i - 1

PROGRAM

E09_STACK.CPP

```cpp
#include <iostream>
using namespace std;

#define MAX 5
int stackArr[MAX];
int top = -1;                                   // -1 = empty

void push(int value)
{
    if (top == MAX - 1)
        cout << "Stack Overflow!" << endl;
    else
    {
        top++;
        stackArr[top] = value;
        cout << value << " pushed into stack." << endl;
    }
}

void pop()
{
    if (top == -1)
        cout << "Stack Underflow!" << endl;
    else
    {
        cout << stackArr[top] << " popped from stack." << endl;
        top--;
    }
}

void peek()
{
    if (top == -1)
        cout << "Stack is empty." << endl;
    else
        cout << "Top element is " << stackArr[top] << endl;
}

void display()
{
    if (top == -1)
        cout << "Stack is empty." << endl;
    else
    {
        cout << "Stack elements (top to bottom):" << endl;
        for (int i = top; i >= 0; i--)
            cout << stackArr[i] << " ";
        cout << endl;
    }
}

int main()
{
    int choice, value;

    do
    {
        cout << "\n1. Push" << endl;
        cout << "2. Pop" << endl;
```

```
        cout << "3. Peek" << endl;
        cout << "4. Display" << endl;
        cout << "5. Exit" << endl;
        cout << "Enter your choice: ";
        cin >> choice;

        if (choice == 1)
        {
            cout << "Enter value to push: ";
            cin >> value;
            push(value);
        }
        else if (choice == 2)
            pop();
        else if (choice == 3)
            peek();
        else if (choice == 4)
            display();
        else if (choice == 5)
            cout << "Program terminated." << endl;
        else
            cout << "Invalid choice!" << endl;
    } while (choice != 5);

    return 0;
}
```

RESULT

The program was executed successfully. 10, 20 and 30 were pushed and popped in the order 30, 20, 10, confirming LIFO. Overflow (sixth push with MAX = 5) and underflow (pop from an empty stack) were detected.

SCREEN (REAL RUN; MENU SHOWN ONCE, IT REPRINTS BEFORE EACH CHOICE)

```
1. Push
2. Pop
3. Peek
4. Display
5. Exit
Enter your choice: 1
Enter value to push: 10
10 pushed into stack.
Enter your choice: 1
Enter value to push: 20
20 pushed into stack.
Enter your choice: 1
Enter value to push: 30
30 pushed into stack.
Enter your choice: 4
Stack elements (top to bottom):
30 20 10
Enter your choice: 3
Top element is 30
Enter your choice: 2
30 popped from stack.
Enter your choice: 4
Stack elements (top to bottom):
20 10
Enter your choice: 2
20 popped from stack.
Enter your choice: 2
```

```
10 popped from stack.
Enter your choice: 2
Stack Underflow!
Enter your choice: 5
Program terminated.
```

Experiment 10 — Queue using an array (enqueue, dequeue)

AIM

To write a program in C++ to implement a queue using an array and perform enqueue and dequeue operations.

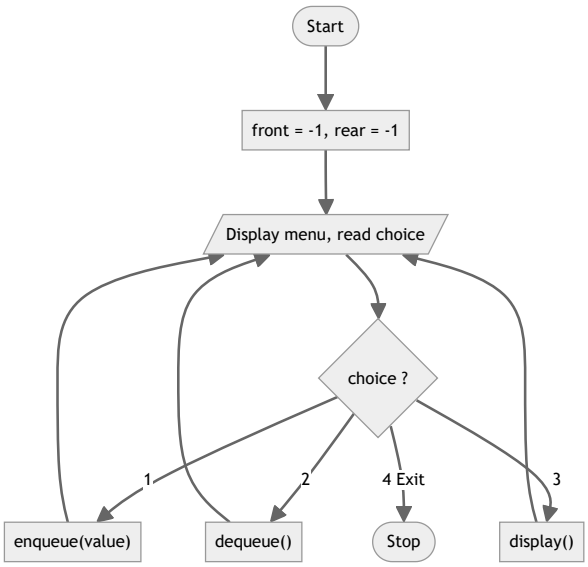
THEORY

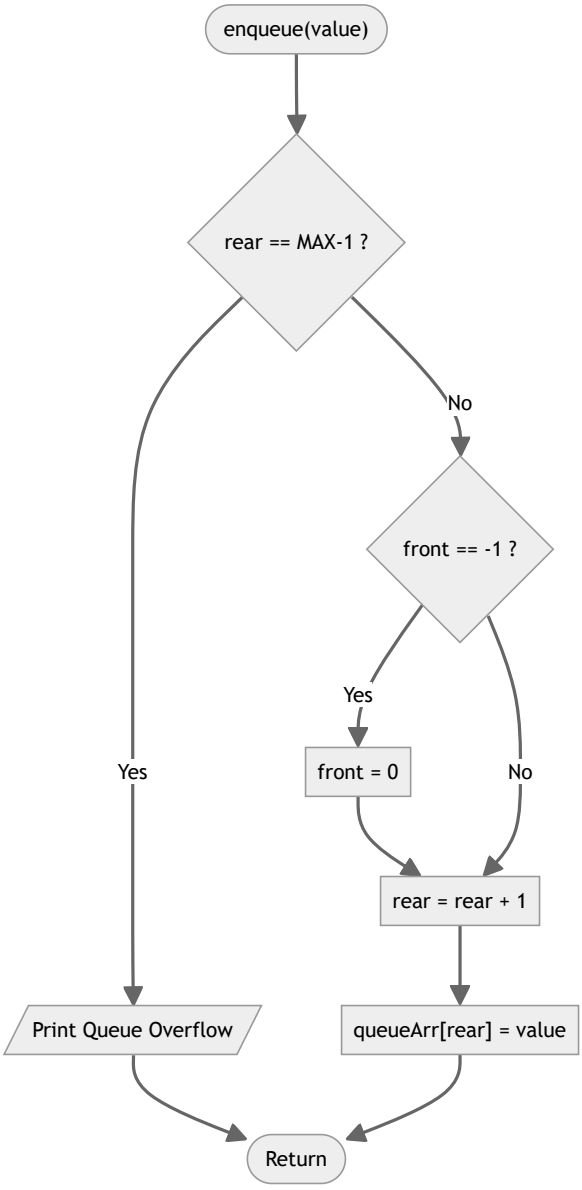
A queue inserts at the rear (enqueue) and deletes from the front (dequeue): FIFO (First In, First Out). front = rear = −1 means empty. Overflow: rear = MAX − 1. Underflow: front = −1. Enqueue and dequeue are O(1). In a linear queue, slots freed at the front are not reused.

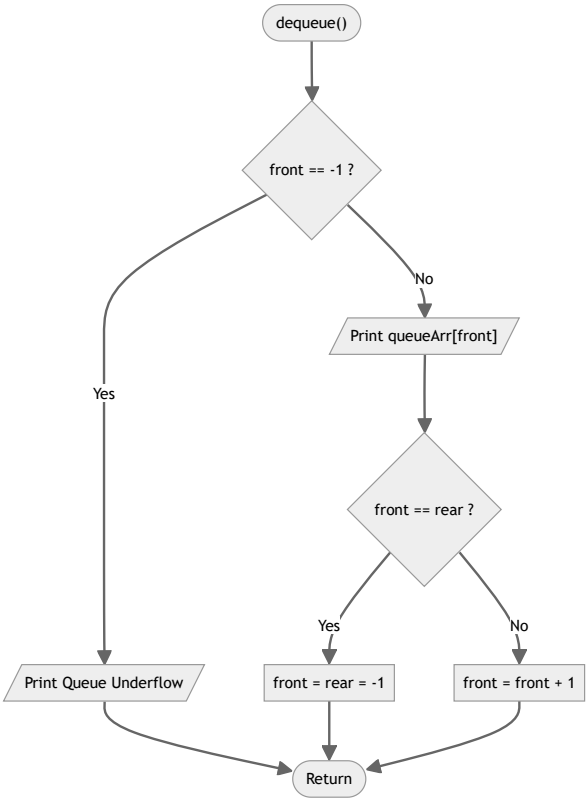
ALGORITHM

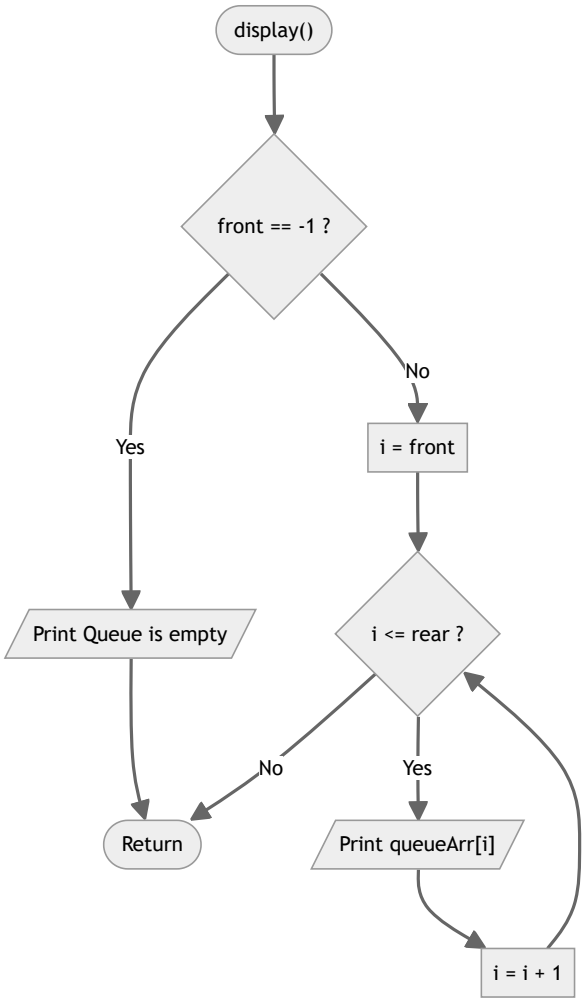
1. Start. Declare `queueArr[MAX]`, front = −1, rear = −1.
2. Display the menu (Enqueue, Dequeue, Display, Exit) and read the choice.
3. **Enqueue:** if rear = MAX − 1, display "Queue Overflow"; else if front = −1 set front = 0; rear = rear + 1; queueArr[rear] = value.
4. **Dequeue:** if front = −1, display "Queue Underflow"; else display queueArr[front]; if front = rear set front = rear = −1, else front = front + 1.
5. **Display:** if front = −1, display "Queue is empty"; else print queueArr[front … rear].
6. Repeat from step 2 until Exit is chosen.
7. Stop.

FLOWCHART


Start
front = -1, rear = -1
Display menu, read choice
choice ?
1
2
4 Exit
3
enqueue(value)
dequeue()
Stop
display()


enqueue(value)
rear == MAX-1 ?
Yes
No
front == -1 ?
Yes
No
front = 0
rear = rear + 1
Print Queue Overflow
queueArr[rear] = value
Return





PROGRAM

E10_QUEUE.CPP

```cpp
#include <iostream>
using namespace std;

#define MAX 5
int queueArr[MAX];
int front = -1;
int rear = -1;

void enqueue(int value)
{
    if (rear == MAX - 1)
    {
        cout << "Queue Overflow!" << endl;
    }
    else
    {
        if (front == -1)                    // first element
            front = 0;
        rear++;
        queueArr[rear] = value;
        cout << value << " inserted into queue." << endl;
    }
}

void dequeue()
```

```
{
    if (front == -1)
    {
        cout << "Queue Underflow!" << endl;
    }
    else
    {
        cout << queueArr[front] << " deleted from queue." << endl;
        if (front == rear)                    // that was the last element
        {
            front = -1;
            rear = -1;
        }
        else
        {
            front++;
        }
    }
}

void display()
{
    if (front == -1)
    {
        cout << "Queue is empty." << endl;
    }
    else
    {
        cout << "Queue elements are:" << endl;
        for (int i = front; i <= rear; i++)
            cout << queueArr[i] << " ";
        cout << endl;
    }
}

int main()
{
    int choice, value;

    do
    {
        cout << "\n1. Enqueue" << endl;
        cout << "2. Dequeue" << endl;
        cout << "3. Display" << endl;
        cout << "4. Exit" << endl;
        cout << "Enter your choice: ";
        cin >> choice;

        if (choice == 1)
        {
            cout << "Enter value to enqueue: ";
            cin >> value;
            enqueue(value);
        }
        else if (choice == 2)
        {
            dequeue();
        }
        else if (choice == 3)
        {
            display();
        }
        else if (choice == 4)
```

```
        {
            cout << "Program terminated." << endl;
        }
        else
        {
            cout << "Invalid choice!" << endl;
        }
    } while (choice != 4);

    return 0;
}
```

RESULT

The program was executed successfully. 10, 20 and 30 were enqueued (display: 10 20 30); dequeue removed 10 (display: 20 30), confirming FIFO. Overflow and underflow conditions were handled.

SCREEN (REAL RUN; MENU SHOWN ONCE, IT REPRINTS BEFORE EACH CHOICE)

```
1. Enqueue
2. Dequeue
3. Display
4. Exit
Enter your choice: 1
Enter value to enqueue: 10
10 inserted into queue.
Enter your choice: 1
Enter value to enqueue: 20
20 inserted into queue.
Enter your choice: 1
Enter value to enqueue: 30
30 inserted into queue.
Enter your choice: 3
Queue elements are:
10 20 30
Enter your choice: 2
10 deleted from queue.
Enter your choice: 3
Queue elements are:
20 30
Enter your choice: 4
Program terminated.
```

Experiment 11 — Singly linked list (insertion, deletion, traversal)

AIM

To write a program in C++ to implement a singly linked list and perform insertion, deletion and traversal operations.

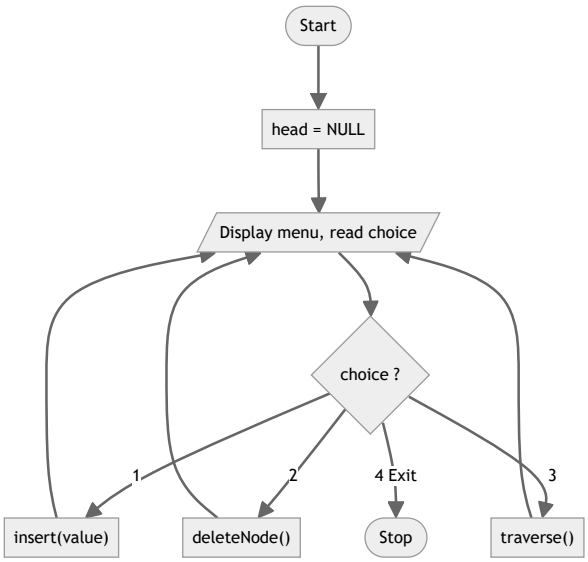
THEORY

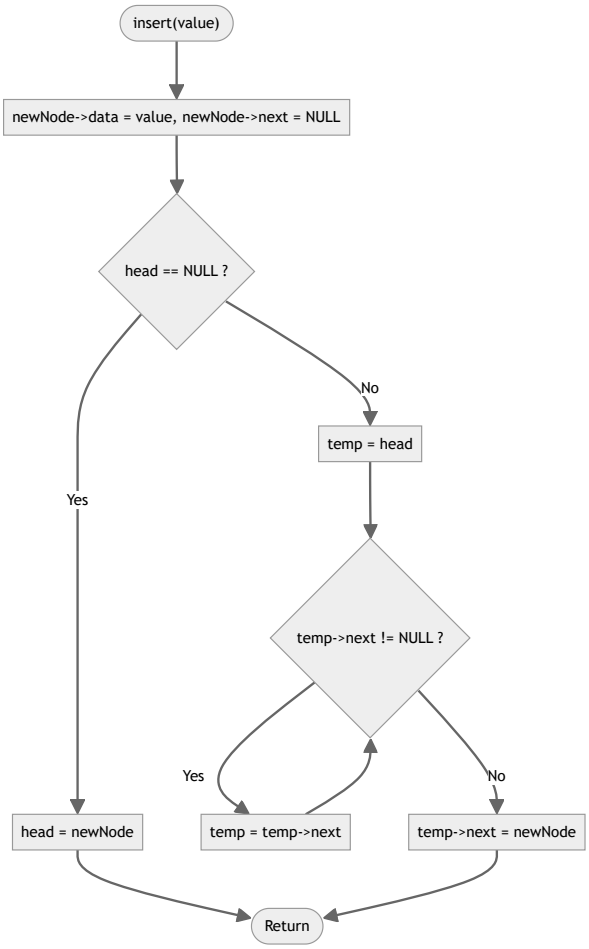
A singly linked list is a chain of nodes, each holding data and a `next` pointer. `head` points to the first node, and the last node points to NULL. Nodes are created with `new` and freed with `delete`. Insert at end O(n), delete at beginning O(1), traversal O(n).

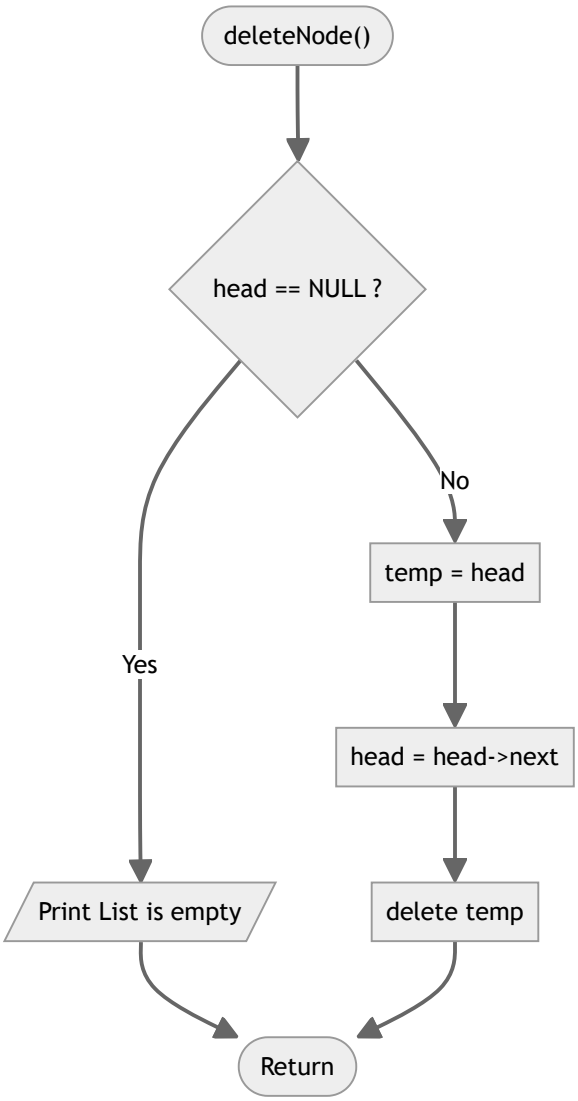
ALGORITHM

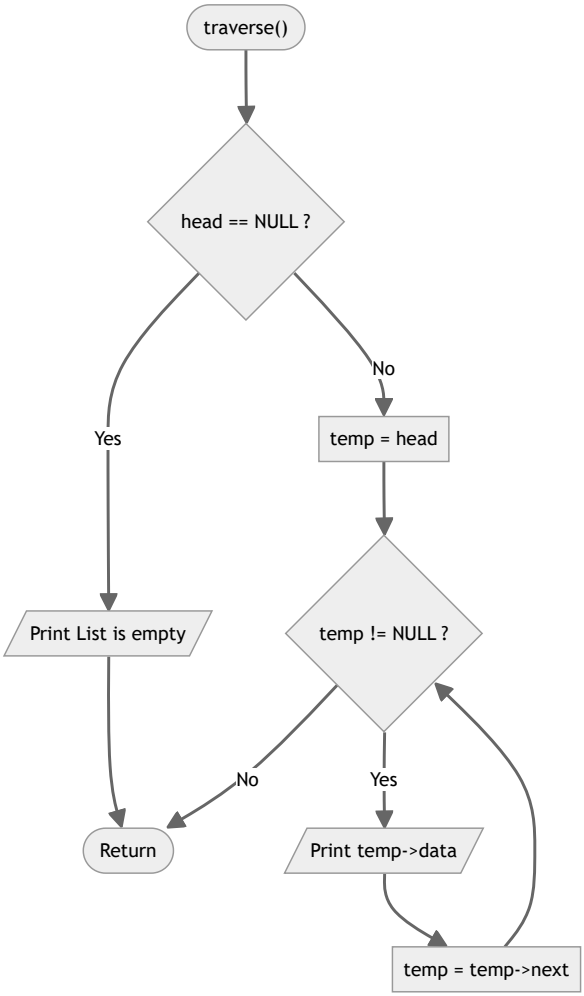
1. Start. Define `struct Node { int data; Node *next; }`; head = NULL.
2. Display the menu (Insert, Delete, Traverse, Exit) and read the choice.
3. **Insert (end):** create newNode with data = value, next = NULL. If head = NULL, head = newNode; else temp = head, move temp until temp→next = NULL, then temp→next = newNode.
4. **Delete (beginning):** if head = NULL, display "List is empty"; else temp = head, display temp→data, head = head→next, delete temp.
5. **Traverse:** if head = NULL, display "List is empty"; else temp = head; while temp ≠ NULL, display temp→data and set temp = temp→next.
6. Repeat from step 2 until Exit is chosen.
7. Stop.

FLOWCHART


Start
head = NULL
Display menu, read choice
choice ?
1
2
4 Exit
3
insert(value)
deleteNode()
Stop
traverse()


insert(value)
newNode->data = value, newNode->next = NULL
head == NULL ?
Yes
No
temp = head
temp->next != NULL ?
Yes
No
head = newNode
temp = temp->next
temp->next = newNode
Return





PROGRAM

E11_SINGLY_LINKED_LIST.CPP

```cpp
#include <iostream>
using namespace std;

struct Node
{
    int data;
    Node *next;
};

Node *head = NULL;                          // empty list

// Insert at the end
void insert(int value)
{
    Node *newNode = new Node;               // ask for memory for one node
    newNode->data = value;
    newNode->next = NULL;

    if (head == NULL)
    {
        head = newNode;
    }
```

```
    else
    {
        Node *temp = head;
        while (temp->next != NULL)          // walk to the last node
            temp = temp->next;
        temp->next = newNode;
    }
    cout << value << " inserted into list." << endl;
}

// Delete from the beginning
void deleteNode()
{
    if (head == NULL)
    {
        cout << "List is empty." << endl;
    }
    else
    {
        Node *temp = head;
        cout << temp->data << " deleted from list." << endl;
        head = head->next;                  // move head first...
        delete temp;                        // ...then free the old first node
    }
}

void traverse()
{
    if (head == NULL)
    {
        cout << "List is empty." << endl;
    }
    else
    {
        Node *temp = head;
        cout << "Linked List: ";
        while (temp != NULL)
        {
            cout << temp->data << " ";
            temp = temp->next;              // without this line: infinite loop
        }
        cout << endl;
    }
}

int main()
{
    int choice, value;

    do
    {
        cout << "\n1. Insert" << endl;
        cout << "2. Delete" << endl;
        cout << "3. Traverse" << endl;
        cout << "4. Exit" << endl;
        cout << "Enter your choice: ";
        cin >> choice;

        if (choice == 1)
        {
            cout << "Enter value to insert: ";
            cin >> value;
            insert(value);
```

```
        }
        else if (choice == 2)
        {
            deleteNode();
        }
        else if (choice == 3)
        {
            traverse();
        }
        else if (choice == 4)
        {
            cout << "Program terminated." << endl;
        }
        else
        {
            cout << "Invalid choice!" << endl;
        }
    } while (choice != 4);

    return 0;
}
```

RESULT

The program was executed successfully. Inserting 10, 20 and 30 gave the list 10 → 20 → 30; deleting from the beginning removed 10, leaving 20 → 30.

SCREEN (REAL RUN; MENU SHOWN ONCE, IT REPRINTS BEFORE EACH CHOICE)

```
1. Insert
2. Delete
3. Traverse
4. Exit
Enter your choice: 1
Enter value to insert: 10
10 inserted into list.
Enter your choice: 1
Enter value to insert: 20
20 inserted into list.
Enter your choice: 1
Enter value to insert: 30
30 inserted into list.
Enter your choice: 3
Linked List: 10 20 30
Enter your choice: 2
10 deleted from list.
Enter your choice: 3
Linked List: 20 30
Enter your choice: 4
Program terminated.
```

Experiment 12 — Circular linked list (insertion, deletion, traversal)

AIM

To write a program in C++ to implement a circular linked list and perform insertion, deletion and traversal operations.

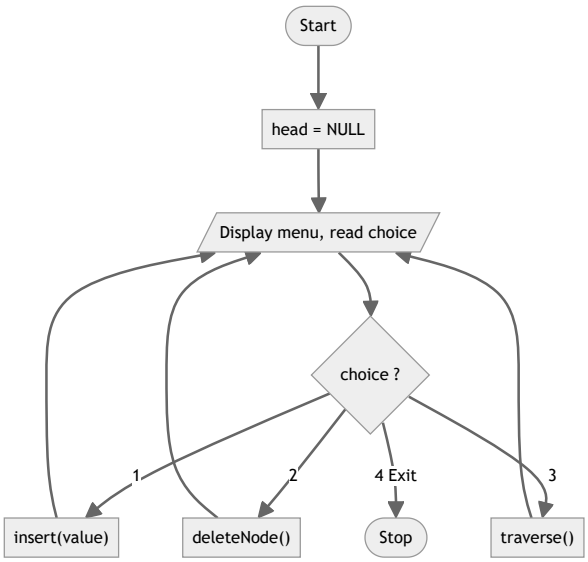
THEORY

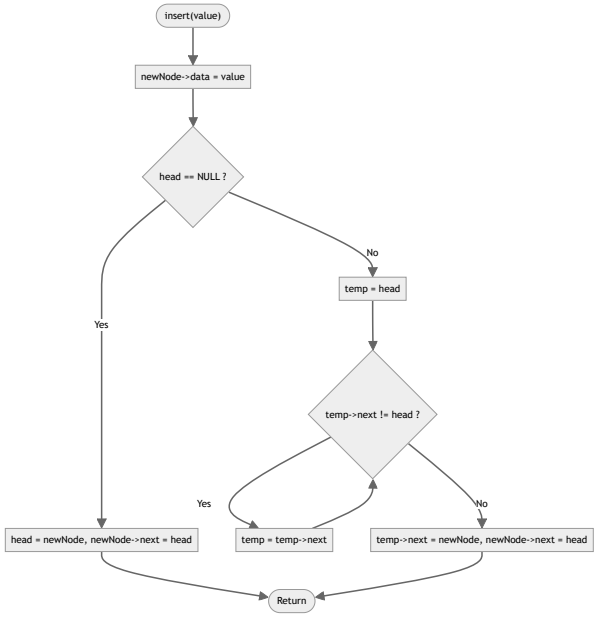
In a circular linked list the last node points back to head instead of NULL (a single node points to itself). The last node is the one with `next == head`, and traversal uses a do-while loop until it returns to head. Insert at end O(n), delete at beginning O(n) (the last node must be re-linked), traversal O(n).

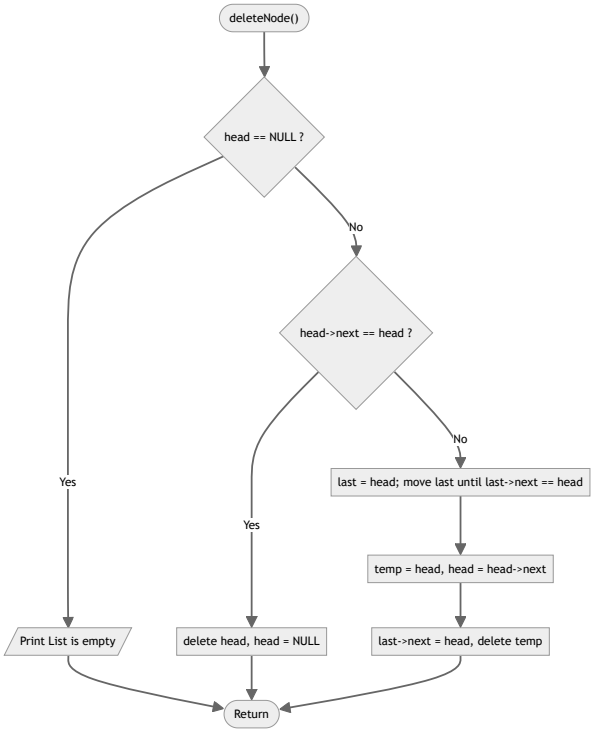
ALGORITHM

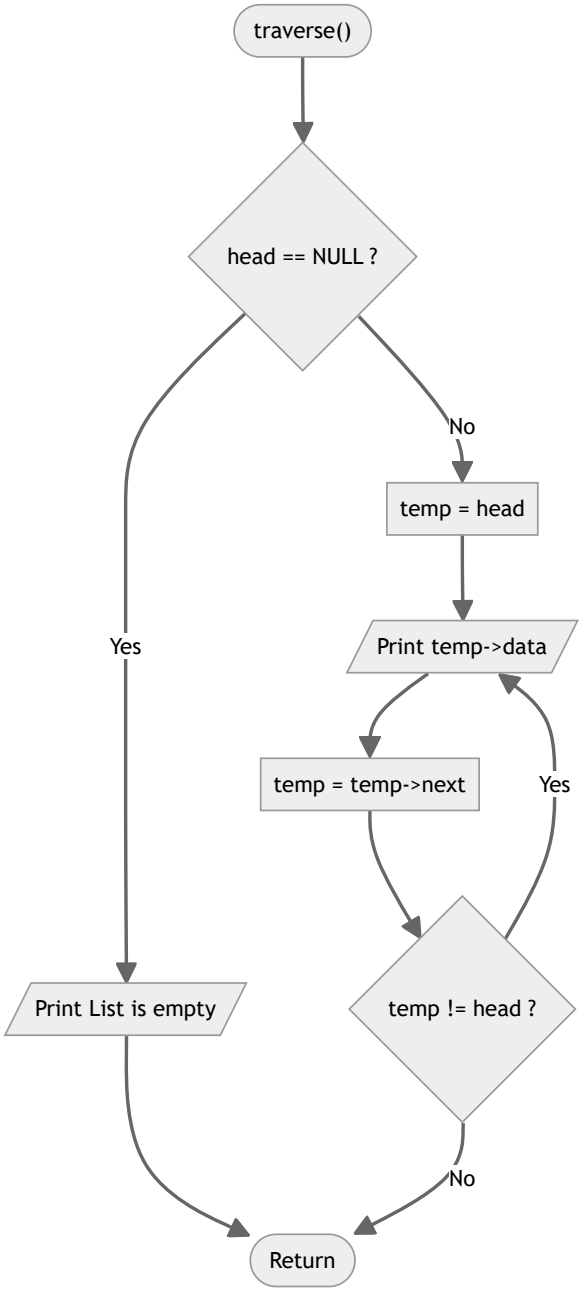
1. Start. Define `struct Node { int data; Node *next; }`; head = NULL.
2. Display the menu (Insert, Delete, Traverse, Exit) and read the choice.
3. **Insert (end):** create newNode with data = value. If head = NULL, head = newNode and newNode→next = head; else move temp from head until temp→next = head, then temp→next = newNode and newNode→next = head.
4. **Delete (beginning):** if head = NULL, display "List is empty"; else if head→next = head, delete head and set head = NULL; else find last (last→next = head), temp = head, head = head→next, last→next = head, delete temp.
5. **Traverse:** if head = NULL, display "List is empty"; else temp = head; do { display temp→data; temp = temp→next } while temp ≠ head.
6. Repeat from step 2 until Exit is chosen.
7. Stop.

FLOWCHART


Start
head = NULL
Display menu, read choice
choice ?
1
2
4 Exit
3
insert(value)
deleteNode()
Stop
traverse()


insert(value)
newNode->data = value
head == NULL ?
Yes
No
temp = head
temp->next != head ?
Yes
No
head = newNode, newNode->next = head
temp = temp->next
temp->next = newNode, newNode->next = head
Return





PROGRAM

E12_CIRCULAR_LINKED_LIST.CPP

```
#include <iostream>
using namespace std;

struct Node
{
    int data;
    Node *next;
};

Node *head = NULL;

// Insert at the end
void insert(int value)
{
    Node *newNode = new Node;
    newNode->data = value;
```

```
    if (head == NULL)
    {
        head = newNode;
        newNode->next = head;               // one node points to itself
    }
    else
    {
        Node *temp = head;
        while (temp->next != head)          // last node is the one pointing to head
            temp = temp->next;
        temp->next = newNode;
        newNode->next = head;               // close the circle
    }
    cout << value << " inserted into list." << endl;
}

// Delete from the beginning
void deleteNode()
{
    if (head == NULL)
    {
        cout << "List is empty." << endl;
    }
    else if (head->next == head)            // only one node
    {
        cout << head->data << " deleted from list." << endl;
        delete head;
        head = NULL;
    }
    else
    {
        Node *temp = head;
        Node *last = head;
        while (last->next != head)          // find the last node
            last = last->next;

        cout << head->data << " deleted from list." << endl;
        head = head->next;
        last->next = head;                  // last now points to the new head
        delete temp;
    }
}

void traverse()
{
    if (head == NULL)
    {
        cout << "List is empty." << endl;
    }
    else
    {
        Node *temp = head;
        cout << "Circular Linked List: ";
        do                                  // do-while: print head before testing
        {
            cout << temp->data << " ";
            temp = temp->next;
        } while (temp != head);
        cout << endl;
    }
}
```

```
int main()
{
    int choice, value;

    do
    {
        cout << "\n1. Insert" << endl;
        cout << "2. Delete" << endl;
        cout << "3. Traverse" << endl;
        cout << "4. Exit" << endl;
        cout << "Enter your choice: ";
        cin >> choice;

        if (choice == 1)
        {
            cout << "Enter value to insert: ";
            cin >> value;
            insert(value);
        }
        else if (choice == 2)
        {
            deleteNode();
        }
        else if (choice == 3)
        {
            traverse();
        }
        else if (choice == 4)
        {
            cout << "Program terminated." << endl;
        }
        else
        {
            cout << "Invalid choice!" << endl;
        }
    } while (choice != 4);

    return 0;
}
```

RESULT

The program was executed successfully. Inserting 10, 20 and 30 gave the circular list 10 → 20 → 30 → (back to 10); deleting from the beginning removed 10, leaving 20 → 30 → (back to 20). The last node always pointed back to head.

SCREEN (REAL RUN; MENU SHOWN ONCE, IT REPRINTS BEFORE EACH CHOICE)

```
1. Insert
2. Delete
3. Traverse
4. Exit
Enter your choice: 1
Enter value to insert: 10
10 inserted into list.
Enter your choice: 1
Enter value to insert: 20
20 inserted into list.
Enter your choice: 1
Enter value to insert: 30
30 inserted into list.
```

```
Enter your choice: 3
Circular Linked List: 10 20 30
Enter your choice: 2
10 deleted from list.
Enter your choice: 3
Circular Linked List: 20 30
Enter your choice: 4
Program terminated.
```